NS3SIONNA: MIMO-ABSTRAKTIONSMODELL FÜR IEEE 802.11AX/BE

Stand: 1. August 2026, Unterphase 3.1 abgeschlossen

Arbeitszweig: feature/su-mimo-minimal

Basis: ns-3.40, ns3sionna c896b858; Auftragsbasis 46d73fa; Sionna 1.2.1

1. ZIEL UND ABGRENZUNG

Sionna RT berechnet die komplexe, frequenzselektive
Kanalübertragungsmatrix. Die eigentliche Link-Abstraktion, also
Post-Processing-SINR, Effective-SINR, MCS/PER und später die Auswahl von
Precoding und MU-MIMO, wird ausschließlich in ns-3 ausgeführt.

Die Entwicklung erfolgt in kleinen, einzeln testbaren Schritten. Der aktuelle
Stand umfasst die Unterphasen 1.1 bis 1.4, die vollständige Phase 2 sowie
Unterphase 3.1:
den numerischen SU-MIMO-Kern, konfigurierbare Sionna-Arrays, einen
dimensionssicheren MIMO-Datenvertrag, explizit kalibrierbare
802.11ax-EESM-/PER-Abbildung und die
schmale Einkopplung in den SpectrumWifiPhy-Empfangspfad. SionnaMimoHelper legt
den High-Level-API-Vertrag und dessen zentrale Validierung fest und wendet ihn
nun konsistent auf Sionna-IPC, Phase-1-Linkabstraktion und SpectrumWifiPhy an.
Der MCS-bezogene ApplyForHeMcs-Aufruf prüft eine für SU-MIMO erforderliche
EESM-Kalibrierung vor jeder Zustandsänderung. Das reproduzierbare Beispiel
konfiguriert 1x1 und Vollrang-2x2 ausschließlich über SionnaMimoHelper und
deckt mit dem deterministischen Kanalpfad SpectrumWifiPhy end-to-end ab.
Integrationsnahe Negativläufe prüfen widersprüchliche NSS, nicht unterstützte
Modi und eine CFR-/PHY-Dimensionsverletzung. Ein dimensionssicherer,
benutzergegebener Precoder unterstützt nun frequenzflaches und
frequenzselektives festes W[k], einschließlich kontrollierter Streamauswahl
mit N_s < N_tx. MRT/MRC, SVD, CSI-Feedback, Beamforming und MU-MIMO wurden
nicht begonnen.

2. REPRODUZIERBARE AUSGANGSBASIS

Die lokale Referenzplattform ist CPU-only:

- Ubuntu 24.04, GCC 13.3, CMake 3.28
- ns-3.40 mit Tests und Beispielen
- ZeroMQ 4.3.5 und Protocol Buffers 3.21.12
- Python 3.12.12, Sionna 1.2.1, NumPy 1.26.4
- TensorFlow erkennt keine GPU und verwendet den CPU-Pfad

Ohne explizite Arraykonfiguration verwendet der Server weiterhin exakt die
bisherige 1x1-Sende- und Empfangsantenne. Die normalisierte SISO-CFR erfüllt

$$
\frac{1}{N_\mathrm{sc}}\sum_k |h[k]|^2 = 1,
$$

während der Wideband-Pfadverlust separat in dB übertragen wird. Unterphase 1.2
verallgemeinert diese Trennung auf alle Antennenpaare: H[k] enthält
Small-Scale-Fading und der skalare Wideband-Verlust bestimmt die Referenz-SNR.

3. ENTWICKLUNGSPLAN

PHASE 0: BASELINE UND MESSVERTRAG

1. Versionen und CPU-Ausführung reproduzierbar festhalten.

2. Bestehende SISO-Import-, Unit- und Beispieltests ausführen.
3. Einheiten, Antennenreihenfolge, CFR-Normalisierung und
   Subcarrier-Reihenfolge als Datenvertrag festlegen.

Abnahmekriterium: SISO-Verhalten bleibt unverändert und alle alten Tests laufen.

  PHASE 1: BELASTBARES SU-MIMO-MINIMALMODELL

  UNTERPHASE 1.1: NUMERISCHER KERN (IMPLEMENTIERT)

  - CFR-Tensor H[k] mit expliziten Dimensionen N_sc x N_rx x N_tx.
  - Open-Loop Spatial Multiplexing, ein Stream je Sendeantenne.
  - Gleiche Gesamtleistungsaufteilung über N_s = N_tx Streams.
  - Perfekte CFR-Kenntnis am Empfänger und linearer MMSE-Detektor.
  - Post-MMSE-SINR pro Subcarrier und Stream.
  - Mutual-Information-basierte Kompression über Subcarrier.
  - Analytische Unit-Tests für SISO, orthogonale 2x2-Kanäle und
  frequenzselektive Kanäle.

Abnahmekriterium: analytische Werte werden mit einer Toleranz von 1e-10
reproduziert.

  UNTERPHASE 1.2: MIMO-DATENVERTRAG SIONNA -> NS-3 (IMPLEMENTIERT)

  - Antennendimensionen rückwärtskompatibel in message.proto ergänzen.
  - Flache Reihenfolge sc/rx/tx verbindlich festlegen.
  - Sionna-Arrays konfigurierbar machen und die CFR ohne Verlust der
  Antennendimensionen serialisieren.
  - CacheEntry auf MimoChannelMatrix umstellen.
  - Python-Shape-Tests und C++-Deserialisierungstests hinzufügen.

Abnahmekriterium: 1x1 ist binär/funktional kompatibel; synthetische 2x2-Werte
kommen indexgenau in ns-3 an.

  UNTERPHASE 1.3: 802.11AX-LINK-ABSTRAKTION (IMPLEMENTIERT)

  - Referenz-SNR aus Sendeleistung, Wideband-Verlust, Bandbreite,
  Rauschzahl und thermischem Rauschen berechnen.
  - Effektive SINR mit einer explizit kalibrierbaren EESM-Abbildung je MCS
  bestimmen; ohne belegte Kalibrierung existieren bewusst keine Default-Betas.
  - Vorhandene ns-3 ErrorRateModel-Schnittstellen soweit möglich wiederverwenden.
  - SISO-Parität und AWGN-Kurven gegen ns-3 validieren.

Abnahmekriterium: PER-vs-SNR-Kurven sind monoton, reproduzierbar und im
SISO-Grenzfall deckungsgleich zur festgelegten Referenz. Alle Kriterien wurden
mit den in Abschnitt 7 beschriebenen automatischen Tests erfüllt.

  UNTERPHASE 1.4: WIFIPHY-INTEGRATION UND BEISPIEL (IMPLEMENTIERT)

  - Nur den schmalsten notwendigen Empfangspfad in SpectrumWifiPhy anbinden.
  - NSS aus WifiTxVector verwenden und ungültige Antennenkonfigurationen früh
  ablehnen.
  - Beispiel ns3_wifi_sumimo_minimal.cc mit 1x1/2x2-Vergleich anlegen.
  - Durchsatz, PER, Stream-SINR und Kanalrang ausgeben.

Abnahmekriterium: 2x2 gewinnt auf gut konditionierten Kanälen und kollabiert
auf Rang-1-Kanälen nachvollziehbar gegen 1x1.
Die automatisierten Ergebnisse in Abschnitt 7 erfüllen dieses Kriterium.

  PHASE 2: HIGH-LEVEL-API

Geplant ist ein SionnaMimoHelper, der mindestens folgende Aspekte gemeinsam konfiguriert:

- Arraygeometrie: Zeilen, Spalten, Elementabstand und Polarisation
- Ausrichtung: Azimut, Elevation und Rollwinkel
- Betriebsart: SISO, SU-MIMO Spatial Multiplexing, SU-Beamforming, MU-MIMO
- Zahl der TX-/RX-Antennen, maximale NSS und Empfängertyp
- Kanalnormalisierung, Rauschzahl und Effective-SINR-Modell

Die API validiert Wi-Fi-, Sionna- und Abstraktionsparameter zentral, sodass Beispiele keine internen Objekte manuell verdrahten müssen.

Phase 2 wird in drei Unterphasen nach API-Vertrag, interner Integration und Benutzbarkeit getrennt. Damit bleiben API-Entscheidungen ohne Änderung des PHY-Verhaltens prüfbar, während Kompatibilitäts- und End-to-End-Regressionen den nachfolgenden Integrationsschritten eindeutig zugeordnet werden können.

UNTERPHASE 2.1: API-VERTRAG (IMPLEMENTIERT)

- SionnaMimoHelper sowie Konfigurationstypen, dokumentierte Defaults und unterstützte Betriebsarten definieren.
- Arraygeometrie, knotenspezifische Ausrichtung, TX-/RX-Antennenzahlen, maximale NSS, Empfängertyp, Kanalnormalisierung, Rauschzahl und Effective-SINR-Modell in einem konsistenten Vertrag zusammenführen.
- Zentrale Validierung für vollständige, endliche und untereinander konsistente Parameter bereitstellen.
- In Phase 2 nur SISO und SU-MIMO Spatial Multiplexing als funktionale Modi unterstützen. Beamforming und MU-MIMO werden entweder noch nicht angeboten oder brechen, falls sie für API-Stabilität bereits repräsentiert werden, eindeutig als nicht unterstützt ab.
- Reine API-Unit-Tests für Defaults sowie gültige und ungültige Konfigurationen hinzufügen, ohne das bestehende PHY-Verhalten zu ändern.

Abnahmekriterium: Der API-Vertrag ist unabhängig von einer Simulation testbar, rückwärtskompatible Defaults sind festgeschrieben und kein Modus aus Phase 3 oder 4 erscheint fälschlich funktionsfähig.

UNTERPHASE 2.2: INTEGRATION (IMPLEMENTIERT)

- SionnaMimoHelper mit den vorhandenen Phase-1-Komponenten für Kanal, Link Budget, MMSE, EESM und SpectrumWifiPhy verbinden.
- Knotenbezogene Arraygeometrie und Ausrichtung über den Sionna-IPC- und Serverpfad anwenden.
- Antennenzahlen, maximale Spatial Streams, Rauschzahl, Empfängertyp, Normalisierung und Effective-SINR-Konfiguration konsistent an die beteiligten ns-3-Objekte weiterreichen.
- Den bisherigen SionnaHelper-Pfad und die globalen 1x1-Defaults unverändert nutzbar halten.
- Integrationstests ergänzen, welche manuelle Phase-1-Konfiguration und Helper-Konfiguration für 1x1 und 2x2 vergleichen.

Abnahmekriterium: Äquivalente manuelle und High-Level-Konfigurationen liefern dieselben Phase-1-Ergebnisse; bestehende 1x1-Beispiele und Tests bleiben unverändert erfolgreich.

UNTERPHASE 2.3: BENUTZBARKEIT UND VALIDIERUNG (IMPLEMENTIERT)

- Minimale 1x1- und 2x2-Beispiele ausschließlich über die High-Level-API anlegen.
- Integrationsnahe Negativtests für widersprüchliche Laufzeitparameter und Fehlerweitergabe vom Helper bis zum PHY ergänzen.

- End-to-End-Tests vom Helper über Sionna beziehungsweise den
  deterministischen Kanalpfad bis SpectrumWifiPhy ausführen.
- API, Defaults, unterstützte Modi, Fehlermeldungen und Migration von der
  manuellen Konfiguration dokumentieren.

Abnahmekriterium: Die 1x1-/2x2-Beispiele laufen ohne manuelle Verdrahtung
interner Objekte, ungültige Konfigurationen brechen früh und eindeutig ab und
Dokumentation sowie PDF beschreiben den abgeschlossenen Stand von Phase 2.
Die helperbasierten End-to-End- und Laufzeit-Negativtests in Abschnitt 7
erfüllen dieses Kriterium.

PHASE 3: SU-MIMO MIT PRECODING UND BEAMFORMING

Sionna liefert weiterhin ausschließlich die komplexe CFR H[k]. Auswahl und
Anwendung des Precoding W[k], Streamauswahl, Leistungsaufteilung und
CSI-Zeitbezug bleiben vollständig in ns-3. Die bestehende skalare Grenze zu
SpectrumWifiPhy bleibt erhalten: Erst nach der matrixwertigen Kanal-,
Precoding- und Detektorberechnung wird auf eine Effective-SINR beziehungsweise
äquivalente Empfangs-PSD komprimiert.

Phase 3 ist inhaltlich größer als Phase 1 oder Phase 2 jeweils für sich: Sie
erweitert gleichzeitig Datenvertrag, numerischen Kern, High-Level-API und
PHY-Integration und führt zusätzlich rückgemeldete sowie zeitabhängige CSI
ein. Die folgenden fünf Unterphasen beschreiben deshalb vertikale,
abnehmbare Funktionen; die einzelnen Aufzählungspunkte bleiben interne
Implementierungsschritte. Unterphasen 3.1 bis 3.3 bilden den idealen
Perfect-CSI-Kern, Unterphase 3.4 ist der eigene Realismus- und
Standardisierungsblock und Unterphase 3.5 schließt die Phase end-to-end ab.

UNTERPHASE 3.1: FESTES PRECODING ALS VOLLSTÄNDIGE GRUNDLAGE (IMPLEMENTIERT)

- Einen dimensionssicheren Vertrag für
  W[k] in C^(N_tx x N_s) mit verbindlicher sc/tx/stream-Reihenfolge
  definieren; frequenzflache Matrizen werden explizit auf alle Subcarrier
  abgebildet.
- Das Signalmodell auf
  y[k] = sqrt(P/N_s) H[k] W[k] x[k] + n[k] verallgemeinern und den bestehenden
  MMSE-Kern auf den effektiven Kanal H_eff[k] = H[k]W[k] anwenden.
- N_s unabhängig von N_tx im Bereich 1 <= N_s <= min(N_tx, N_rx) führen.
- Die Leistungsnormierung ||W[k]||_F^2 = N_s festlegen, sodass P weiterhin die
  gesamte Sendeleistung bezeichnet und kein unbeabsichtigter Precoding-Gewinn
  entsteht.
- Analytische Tests für SISO, Identitäts-Precoding, Streamauswahl,
  komplexwertige feste Matrizen und Dimensions-/Normierungsfehler ergänzen.

Numerisches Zwischenkriterium: W[k] = I reproduziert den Phase-1-Kern
innerhalb 1e-10;
synthetische H[k]W[k]-Referenzen liefern indexgenau dieselben
Post-MMSE-SINRs und die Gesamtleistung bleibt invariant.

Integrationsschritte innerhalb derselben Unterphase:

- Benutzergegebenes frequenzflaches und frequenzselektives W[k] durch
  SionnaMimoHelper, Link-Abstraktion und SpectrumWifiPhy-Adapter führen.
- WifiTxVector::GetNss an die Spaltenzahl von W[k] statt weiterhin zwingend
  an N_tx koppeln und N_s < N_tx erstmals kontrolliert zulassen.
- Subcarrierzahl, Antennenmaße, NSS, endliche Koeffizienten und
  Leistungsnormierung vor jeder Zustandsänderung zentral validieren.
- Ohne explizites Precoding den bisherigen SISO-/Open-Loop-Pfad byte- und
  funktional kompatibel halten.
- API-, Adapter- und End-to-End-Tests für 1x1, 2x2-Identität sowie

N_tx = 2, N_rx = 2, N_s = 1 ergänzen.

Abnahmekriterium: Manuell und über SionnaMimoHelper konfiguriertes festes W[k] liefern dieselben SINR-, EESM- und PER-Ergebnisse; alle Phase-2-Tests bleiben unverändert erfolgreich. Die analytischen, Adapter- und SpectrumWifiPhy-End-to-End-Ergebnisse in Abschnitt 7 erfüllen dieses Kriterium.

UNTERPHASE 3.2: EIN-STREAM-MRT/MRC

- MRT für den eindeutig definierten MISO-Fall N_rx = 1 aus perfekter CFR berechnen und auf Einheitsnorm normieren.
- Für N_tx = 1 die bestehende Ein-Stream-MMSE-Lösung als äquivalente MRC-Kombination ausweisen und validieren.
- Den allgemeinen Fall N_tx > 1 und N_rx > 1 nicht mehrdeutig als MRT/MRC bezeichnen; dessen optimale Ein-Stream-Lösung gehört zur SVD-Unterphase.
- Analytische MISO-/SIMO-Kanäle, gemeinsame Phasendrehung, Nullkanal und erwarteten Arraygewinn testen.

Abnahmekriterium: MRT/MRC erreicht auf analytischen Ein-Stream-Kanälen die Post-Combining-SNR rho ||h||_2^2, bleibt leistungsnormalisiert und fällt im 1x1-Grenzfall exakt auf das bestehende Ergebnis zurück.

UNTERPHASE 3.3: SVD-PRECODING UND LEISTUNGSALLOKATION

- Pro Subcarrier H[k] = U[k] Sigma[k] V^H[k] zerlegen und die N_s stärksten rechten Singulärvektoren bei zunächst gleicher Streamleistung als W[k] verwenden.
- N_s explizit wählen und gegen Antennenzahlen sowie einen dokumentierten numerischen Rangschwellwert validieren.
- Sortierung, Vorzeichen-/Phasenkonvention und Verhalten bei gleichen Singularwerten deterministisch festlegen.
- Diagonale, unitär gedrehte, frequenzselektive und rangdefiziente Referenzkanäle gegen analytische Eigenmode-SINRs testen.

Zwischenkriterium für SVD und Streamauswahl: Die Eigenmode-SINRs stimmen innerhalb 1e-10 mit
den Singularwertreferenzen überein; Rangverlust wird durch eine explizite Streamwahl behandelt und nicht durch versteckte Regularisierung kaschiert.

Optionale Leistungsallokation innerhalb derselben Unterphase:

- Gleiche Leistung als reproduzierbaren Standard beibehalten und Wasserfüllung als explizit auswählbare Forschungsoption ergänzen.
- Den Umfang zunächst auf räumliche SVD-Moden je Subcarrier begrenzen, damit spektrale Maske und Gesamtleistung pro Subcarrier unverändert bleiben.
- Rauschleistung, Referenz-SNR, aktive Moden und die Summe der zugewiesenen Leistung vollständig ausgeben und validieren.
- Analytische Zwei-Moden-Fälle unterhalb und oberhalb der Aktivierungsschwelle sowie Leistungserhaltung und KKT-Bedingungen testen.

Abnahmekriterium der optionalen Erweiterung: Die Leistungsanteile sind endlich, nicht negativ und
summieren sich exakt zur vorgegebenen Gesamtleistung; Wasserfüllung erreicht auf den Referenzkanälen mindestens die ideale Rate der geprüften Gleichleistungsreferenz, ohne Sendeleistung zu erzeugen.

UNTERPHASE 3.4: QUANTISIERTE RÜCKMELDUNG UND VERALTETE CSI

- Vor der Implementierung das konkret unterstützte HE-/EHT-Feedbackformat, Winkel-/Codebookdarstellung, Bitbreiten und Rekonstruktionsreihenfolge

verbindlich dokumentieren; nicht unterstützte Standardvarianten klar abgrenzen.
- Ideales SVD-Precoding, Quantisierung und Rekonstruktion als getrennte, einzeln testbare Schritte modellieren.
- Quantisiertes W[k] erneut leistungsnormalisieren und Feedbackumfang sowie Quantisierungsfehler als Diagnosegrößen bereitstellen.
- Bekannte Vektoren, Grenzwinkel, minimale/maximale Quantisierung und den unquantisierten Grenzfall testen.

Zwischenkriterium für die Quantisierung: Bitgenaue Referenzvektoren werden deterministisch
rekonstruiert, die Precoder bleiben leistungsnormalisiert und feinere Quantisierung verschlechtert die Referenzmetrik auf den Testkanälen nicht.

CSI-Alterung innerhalb derselben Unterphase:

- Die zur Precoderwahl verwendete Schätzung H_hat[k,t_csi] strikt von der bei der Paketübertragung wirksamen CFR H[k,t_tx] trennen.
- CSI-Alter, Aktualisierungsintervall und Feedbackverzögerung explizit und reproduzierbar konfigurieren; perfektes aktuelles CSI bleibt der Nullalter-Grenzfall.
- Richtung, Arraykonfiguration, Precodingmodus und CSI-Zeitstempel in Cache- beziehungsweise Zustandsidentität berücksichtigen.
- Statische Kanäle, kontrollierte Phasen-/Subraumrotationen und wiederholte Läufe mit identischen Seeds testen.

Abnahmekriterium: CSI-Alter null reproduziert das ideale Precoding exakt; kontrolliert veraltete Schätzungen werden nachweislich nur zur Wahl von W verwendet, während SINR und PER stets mit dem aktuellen H berechnet werden.

UNTERPHASE 3.5: BENUTZBARKEIT UND PHASENVALIDIERUNG

- SionnaMimoHelper erst nach erfolgreicher Implementierung die jeweiligen Beamforming-/Precodingmodi als unterstützt melden lassen.
- Das Beispiel ns3_wifi_sumimo_precoding.cc anlegen und Open-Loop, festes Precoding, MRT sowie SVD bei gleicher Gesamtleistung vergleichen; Wasserfüllung, Quantisierung und CSI-Alter werden als explizite Optionen zugeschaltet.
- Durchsatz, PER, Stream-SINR, Kanalrang, Precodingnorm, Leistungsanteile und CSI-Alter maschinenlesbar ausgeben.
- Negative Konfigurationstests und vollständige CPU-End-to-End-Regressionen ergänzen sowie API-Migration und Modellgrenzen dokumentieren.

Abnahmekriterium: Die Beispiele benötigen keine manuelle Verdrahtung interner Objekte, analytisch überlegene Precoder zeigen den erwarteten Gewinn ohne zusätzliche Sendeleistung, ungültige Konfigurationen brechen früh ab und alle Phase-1-/2-Regressionen bleiben erfolgreich.

PHASE 4: MU-MIMO

Unterphasen:

1. mehrere CFR-Matrizen H_u[k] in einem gemeinsamen Zeitstempel
2. Downlink-ZF und regularisiertes ZF
3. Inter-User-Interferenz in der Stream-SINR
4. RU/NSS-Zuordnung für HE/EHT und Scheduler-Schnittstelle
5. Uplink-MU-MIMO

Das Phasenbeispiel ns3_wifi_mumimo.cc vergleicht SU-Round-Robin mit MU-ZF bei unterschiedlicher räumlicher Nutzerkorrelation.

PHASE 5: KALIBRIERUNG, SKALIERUNG UND REPRODUZIERBARKEIT

- PER-Kurven gegen Link-Level-Referenzen kalibrieren.
- Sensitivität gegenüber Antennenabstand, Orientierung, CSI-Alter und Raytracing-Samples untersuchen.
- Cache- und IPC-Speicherbedarf für große N_sc, N_tx und N_rx messen.
- Deterministische Seeds, Ergebnisformate und CI-Matrix für CPU/GPU festlegen.

4. DATENMODELL UND IPC-VERTRAG DER UNTERPHASEN 1.1/1.2

4.1 TENSOR UND FLACHE REIHENFOLGE

Für jeden Subcarrier k ist

$$
\mathbf{H}[k] \in \mathbb{C}^{N_\mathrm{rx}\times N_\mathrm{tx}}.
$$

Der C++-Container und die beiden Protocol-Buffer-Felder csi_real und csi_imag speichern die Werte flach. Der Index für den Eintrag h[k,r,t] lautet

$$
i(k,r,t) = (k N_\mathrm{rx} + r)N_\mathrm{tx} + t.
$$

Damit ist k die langsamste und t die schnellste Dimension. Es gelten die verbindlichen Längen

$$
|\mathrm{frequencies}| = N_\mathrm{sc},
\qquad
|\mathrm{csi_real}| = |\mathrm{csi_imag}|
= N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}.
$$

Die neuen skalaren Felder num_rx_antennas und num_tx_antennas wurden mit den Feldnummern 9 und 10 an RxNodeInfo angehängt. Der Proto3-Standardwert null wird beim Empfänger als eins interpretiert. Für eine neue 1x1-Antwort lässt der Python-Server beide Felder bewusst ungesetzt; ihre serialisierte Darstellung ist dadurch bytegleich zur alten Nachricht. Alte Sender werden als 1x1 gelesen, alte Empfänger ignorieren die unbekannten neuen Felder einer MIMO-Antwort.

4.2 TRANSFORMATION AUS SIONNA

Sionna 1.2.1 liefert die CFR in der Form

$$
H_\mathrm{raw}
[\ell,r,q,t,n,k]
$$

mit Linkindex ℓ, RX-Antenne r, TX-Index q, TX-Antenne t, Zeitindex n und Subcarrier k. Eine ns3sionna-Anfrage enthält genau einen Sender und einen Zeitpunkt. Deshalb wird ohne squeeze-Operation explizit ausgewählt:

$$
H_\ell[r,t,k] =
H_\mathrm{raw}[\ell,r,0,t,0,k].
$$

Vor der Serialisierung wird nach [k,r,t] transponiert und anschließend in

C-Reihenfolge abgeflacht. Dadurch bleiben auch Singleton-Dimensionen sichtbar
und 2x2-Werte landen indexgenau im C++-Container.

4.3 WIDEBAND-VERLUST UND MIMO-NORMALISIERUNG

Für die rohe Link-CFR wird die mittlere Leistung gemeinsam über Subcarrier und
Antennenpaare gebildet:

$$
\bar P =
\frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
\sum_{k=0}^{N_\mathrm{sc}-1}
\sum_{r=0}^{N_\mathrm{rx}-1}
\sum_{t=0}^{N_\mathrm{tx}-1}
|H_\ell[r,t,k]|^2.
$$

Der übertragene skalare Wideband-Verlust und die normalisierte Matrix lauten

$$
L_\mathrm{wb} = -10\log_{10}(\bar P)\ \mathrm{dB},
\qquad
H_\mathrm{norm}[r,t,k] =
\frac{H_\ell[r,t,k]}{\sqrt{\bar P}}.
$$

Damit gilt

$$
\frac{1}{N_\mathrm{sc}N_\mathrm{rx}N_\mathrm{tx}}
\sum_{k,r,t}|H_\mathrm{norm}[r,t,k]|^2 = 1.
$$

Für N_rx=N_tx=1 ist dies exakt die bisherige SISO-Normalisierung. Die Aufteilung
der gesamten Sendeleistung auf N_s Streams bleibt davon getrennt und erfolgt
weiterhin im numerischen Modell mit dem Faktor 1/N_s.

4.4 ARRAYKONFIGURATION UND NS-3-CACHE

SimInitMessage besitzt optionale tx_array- und rx_array-Nachrichten mit Zeilen,
Spalten, vertikalem und horizontalem Abstand in Wellenlängen, Elementmuster und
Polarisation. Ohne Nachricht werden exakt die alten Arrays verwendet:
1x1/tr38901/V am Sender und 1x1/dipole/V am Empfänger. SionnaHelper bietet dafür
die kleinen Methoden ConfigureTransmitArray und ConfigureReceiveArray; die
umfassende knotenbezogene High-Level-API bleibt Phase 2 vorbehalten.

Die Antwortdimensionen werden aus der tatsächlich von Sionna gelieferten Form
abgeleitet und berücksichtigen damit auch die Polarisation. CacheEntry hält die
CFR nun als MimoChannelMatrix. GetPropagationChannelMatrix erhält die
Dimensionen; GetPropagationCSI bleibt als flache, für 1x1 unveränderte
Kompatibilitätsansicht bestehen.

5. SU-MIMO-MINIMALMODELL

5.1 SIGNALMODELL

Ohne Precoding gilt je Subcarrier

$$
\mathbf{y}[k] =
\sqrt{\frac{P}{N_s}}\,\mathbf{H}[k]\mathbf{x}[k] + \mathbf{n}[k],
$$

\]

mit N_s = N_tx, E[x x^H] = I und räumlich weißem Rauschen
E[n n^H] = N I. Die Eingangsgröße

\[
  \rho = \frac{P}{N}
\]

ist die gesamte Referenz-SNR vor der Small-Scale-CFR-Gewichtung. Das Modell
verlangt in dieser Unterphase N_rx >= N_tx. Overloaded MIMO wird erst mit einer
expliziten Streamauswahl zugelassen.

  5.2 LINEARER MMSE-EMPFÄNGER

Mit alpha = rho/N_s und G[k] = H[k]^H H[k] folgt die
Fehlerkovarianz des linearen MMSE-Detektors:

\[
  \mathbf{E}[k] =
  (\mathbf{I} + \alpha\mathbf{G}[k])^{-1}.
\]

Die Post-Processing-SINR des Streams s ist

\[
  \gamma_s[k] = \frac{1}{E_{s,s}[k]} - 1.
\]

Die Formel berücksichtigt gewünschte Leistung, Inter-Stream-Interferenz und
Rauschverstärkung gemeinsam. Sie ist auch für rangdefiziente H numerisch
definiert, weil I + alpha G positiv definit ist.

  5.3 SISO-GRENZFALL

Für N_tx = N_rx = 1 ergibt sich

\[
  E = \frac{1}{1 + \rho |h|^2},
  \qquad
  \gamma = \rho |h|^2.
\]

Damit ist der neue Kern im skalaren Grenzfall exakt kompatibel mit dem
üblichen SISO-SNR-Modell.

  5.4 FREQUENZKOMPRESSION

Vor einer MCS-spezifischen Kalibrierung wird je Stream eine
Mutual-Information-erhaltende AWGN-SNR ausgewiesen:

\[
  \gamma_{\mathrm{eff},s} =
  \exp\left(
    \frac{1}{N_\mathrm{sc}}\sum_k \ln(1+\gamma_s[k])
  \right)-1.
\]

Die mittlere ideale spektrale Effizienz über alle Streams ist

\[

$$
  \eta =
  \frac{1}{N_\mathrm{sc}}
  \sum_k\sum_s \log_2(1+\gamma_s[k]).
$$

Diese MI-Größe bleibt eine Validierungsmetrik und ist keine unmittelbare 802.11-Datenrate oder PER. Die HE-PER-Abbildung verwendet stattdessen die MCS-spezifische EESM-Kompression aus Abschnitt 5.6.

5.5 REFERENZ-SNR AUS DEM LINK BUDGET

Die skalare Wideband-Dämpfung L_wb bleibt von der normalisierten Small-Scale-CFR getrennt. Aus der gesamten Sendeleistung $P_\mathrm{tx,dBm}$ folgen Empfangsleistung und lineare Empfangsleistung

$$
  P_\mathrm{rx,dBm} = P_\mathrm{tx,dBm} - L_\mathrm{wb},
  \qquad
  P_\mathrm{rx} =
  10^{(P_\mathrm{rx,dBm}-30)/10}\ \mathrm{W}.
$$

Mit Rauschbandbreite B in Hz, Temperatur T in Kelvin und Empfängerrauschzahl NF in dB gelten

$$
  N_\mathrm{th} = kTB,
  \qquad
  F = 10^{NF/10},
  \qquad
  N_\mathrm{rx} = FkTB,
$$

und damit

$$
  \rho =
  \frac{P_\mathrm{rx}}{N_\mathrm{rx}}
  =
  \frac{10^{(P_\mathrm{tx,dBm}-L_\mathrm{wb}-30)/10}}
    {kTB\,10^{NF/10}}.
$$

WifiLinkBudget gibt Empfangsleistung, thermisches Rauschen, Empfängerrauschen und ρ separat aus. dB-/Linear- und dBm-/Watt-Umrechnungen delegieren an DbToRatio, RatioToDb und DbmToW aus ns-3.40 wifi-utils. Für $k=1{,}3803\cdot10^{-23}\ \mathrm{J/K}$ wird bewusst derselbe Zahlenwert wie in ns-3.40 InterferenceHelper verwendet. Das ist die Provenienz dieser physikalischen Konstante; es ist keine Fit-Konstante. Anders als InterferenceHelper ist T hier ein expliziter positiver Eingabeparameter und nicht fest auf 290 K gesetzt.

P_tx ist die Gesamtleistung über alle Spatial Streams; die Aufteilung durch N_s erfolgt erst im MMSE-Modell. B ist die Empfänger-Rauschbandbreite. Link Budget und MMSE addieren selbst keine externe Interferenz; der SpectrumWifiPhy-Pfad bewertet sie anschließend skalar gemäß Abschnitt 5.8. Ein zusätzlicher pauschaler RX-Diversity-Gewinn wird nicht aufgeschlagen: Array- und Detektorgewinn sind bereits in $\mathbf H[k]$ und den Post-MMSE-SINRs enthalten.

5.6 MCS-ABHÄNGIGE EESM-KOMPRESSION

Für HE MCS m komprimiert EESM alle $N_\mathrm{sc}N_s$
Post-MMSE-SINRs mit gleicher Gewichtung:

$$
\gamma_{\mathrm{EESM},m}
=
-\beta_m \ln\left(
\frac{1}{N_\mathrm{sc}N_s}
\sum_{k=0}^{N_\mathrm{sc}-1}
\sum_{s=0}^{N_s-1}
\exp\left(-\frac{\gamma_s[k]}{\beta_m}\right)
\right).
$$

$\beta_m>0$ hat wie γ lineare SINR-Einheiten und ist
MCS-abhängig. SetEesmBeta erwartet den linearen Wert; SetEesmBetaDb ist die
dB-Komfortfunktion. Zur Vermeidung numerischen Unterlaufs wird mit
$\gamma_\min=\min_{k,s}\gamma_s[k]$ äquivalent gerechnet:

$$
\gamma_{\mathrm{EESM},m}
=
\gamma_\min
-\beta_m \ln\left(
\frac{1}{N_\mathrm{sc}N_s}
\sum_{k,s}
\exp\left(-\frac{\gamma_s[k]-\gamma_\min}{\beta_m}\right)
\right).
$$

Für einen flachen Kanal mit identischen SINRs ist das Ergebnis unabhängig von
β_m exakt diese SINR. Das ermöglicht die SISO-AWGN-Paritätsprüfung.

Es werden bewusst keine Default-Betas ausgeliefert. Jeder HE MCS von 0 bis 11
muss vor der Nutzung explizit konfiguriert werden. Damit enthält der Code keine
unbelegten Kalibrierkonstanten. Die in analytischen Tests verwendeten positiven
Werte sind ausschließlich Formel-Sentinels und ausdrücklich keine
802.11ax-Kalibrierung. Belastbare β_m müssen später aus dokumentierten
Link-Level-Kurven gefittet und zusammen mit Datensatz, PHY-Konfiguration,
Codeart und Paketlänge versioniert werden. Die bisherige MI-Kompression je
Stream bleibt als modellunabhängige Diagnosemetrik erhalten und wird nicht als
kalibrierte PER-Abbildung ausgegeben.

5.7 ABBILDUNG AUF NS-3-PER

HeSuMimoLinkAbstraction übernimmt WifiTxVector einschließlich HE-MCS, NSS und
Codeart. NSS muss der Zahl der übertragenen Streams entsprechen. Nach EESM wird
die vorhandene Schnittstelle unverändert verwendet:

$$
P_\mathrm{succ} =
\mathrm{ErrorRateModel::GetChunkSuccessRate}
(m,\mathrm{TxVector},\gamma_{\mathrm{EESM},m},8L,N_\mathrm{rx}),
\qquad
PER = 1-P_\mathrm{succ},
$$

wobei L die Paketlänge in Byte ist. ErrorRateModel ist als ns-3-Attribut
und über SetErrorRateModel konfigurierbar. Standard ist
TableBasedErrorRateModel, also das ns-3.40-Defaultmodell für 802.11n/ac/ax.

Die Kalibrierprovenienz bleibt vollständig bei ns-3: Laut
src/wifi/doc/source/wifi-design.rst stammen die AWGN-PER-Tabellen aus
End-to-End-Linksimulationen mit MATLAB WLAN Toolbox, wurden gegen IEEE-TGn-
Ergebnisse validiert und verwenden die Paketlängenextrapolation aus
IEEE P802.11 TGax 11-14/0803r1. BCC besitzt Referenztabellen für 32 und
1458 Byte mit einer 400-Byte-Schwelle und MCS 0 bis 9; MCS 10/11 fällt auf
YansErrorRateModel zurück. LDPC besitzt eine 1458-Byte-Referenz für MCS 0 bis
11. Die neue Abstraktion kopiert oder verändert keine dieser Kurven und fügt
keinen SNR-Offset hinzu. Da sie Erfolgswahrscheinlichkeiten und keine
Zufallsentscheidung ausgibt, ist die Berechnung bei identischen Eingaben
deterministisch.

5.8 EINKOPPLUNG IN SPECTRUMWIFIPHY

Die Integration bleibt auf
SionnaSpectrumPropagationLossModel::DoCalcRxPowerSpectralDensity beschränkt.
Für einen HE-SU-PPDU liest sie N_s ausschließlich mit
WifiTxVector::GetNss. Vor der Link-Abstraktion müssen
die physikalischen TX-/RX-Antennenzahlen gleich den CFR-Dimensionen, die
maximal unterstützten Spatial Streams mindestens N_s und
$N_\mathrm{ant,rx}\geq N_s$ sein. Ohne konfiguriertes Precoding bleibt
zusätzlich unverändert $N_s=N_\mathrm{tx}$. Mit festem Precoding muss
N_s dagegen exakt der Spaltenzahl von W[k] entsprechen. Inkonsistenzen
brechen damit vor MMSE, EESM und Paketempfang ab. Der neue kontrollierte Fall
$N_s<N_\mathrm{tx}$ ist ausschließlich mit explizitem festem W[k] zulässig.

Der SpectrumChannel hat den skalaren Wideband-Pfadverlust bereits auf
params->psd angewendet. Die MIMO-Integration verwendet daher

$$
  P_\mathrm{in} =
  \int S_\mathrm{params}(f)\,\mathrm df \; G_\mathrm{rx}
$$

als bereits pfadverlustbehaftete Empfangsleistung. Der Wideband-Verlust aus
dem Cache wird an dieser Stelle nur noch für den bestehenden Diagnose-Tag
gelesen und nicht erneut multipliziert. Empfangsverstärkung,
Rauschbandbreite, 290 K und Empfängerrauschzahl gehen genau einmal in das
Link Budget ein. Weil ns-3.40 die Rauschzahl nicht lesbar als WifiPhy-Attribut
bereitstellt, verlangt SetReceiverNoiseFigureDb einen expliziten Wert; er muss
mit SpectrumWifiPhy::RxNoiseFigure identisch konfiguriert werden. Ein
fehlender Wert bricht den MIMO-Pfad früh ab.

MMSE und EESM liefern γ_eff. SpectrumWifiPhy erwartet
dagegen weiterhin eine skalare PSD und wendet seine InterferenceHelper-
Berechnung später selbst an. Für das AWGN-Fehlermodell ist dessen interner
Diversity-Faktor

$$
  G_\mathrm{AWGN} =
  \begin{cases}
    N_\mathrm{ant,rx}/N_s, & N_\mathrm{ant,rx}>N_s,\\
    1, & \text{sonst}.
  \end{cases}
$$

Da der MMSE-Schritt den Arraygewinn bereits enthält, wird die äquivalente
skalare Empfangsleistung auf

$$

$$
  P_\mathrm{eq} =
  \frac{\gamma_\mathrm{eff} P_N}{G_\mathrm{AWGN}},
  \qquad P_N = kTB\,10^{NF/10}
$$

gesetzt. InterferenceHelper erhält damit bei zusätzlicher skalarer Interferenz I

$$
  \gamma_\mathrm{phy}
  = \frac{P_\mathrm{eq}}{P_N+I}G_\mathrm{AWGN}
  = \gamma_\mathrm{eff}\frac{P_N}{P_N+I}.
$$

Ohne Interferenz gilt exakt $\gamma_\mathrm{phy}=\gamma_\mathrm{eff}$; Pfadverlust, thermisches Rauschen, Rauschzahl und Arraygewinn werden nicht doppelt angewendet. Zeitvariable Interferenz wirkt weiterhin über SpectrumWifiPhy, jedoch als skalare Nachbewertung und nicht als subcarrierabhängige Interferenzkovarianz im MMSE-Detektor.

Der bisherige 1x1-Pfad bleibt unverändert skalar und benötigt kein EESM-Beta. Nicht-HE-Kontrollrahmen einer MIMO-PHY verwenden während Phase 1 das erste Antennenpaar, damit Beacon, Association und ACK ohne vorweggenommene Streamauswahl funktionieren. Der letzte Guard-Bin und das bestehende CFR-Tag-Format bleiben im 1x1-Fall kompatibel. Für automatisierte Beispiele kann eine normalisierte feste CFR den Sionna-Cache ersetzen; dieser deterministische Test-Hook ändert den Produktionspfad nicht. Der MimoRx-Trace meldet PPDU-UID, Knoten, Effective-SINR, PER und den kleinsten Kanalrang über alle Subcarrier.

Das Beispiel ns3_wifi_sumimo_minimal.cc erzeugt drei flache, normierte Kanäle bei HE MCS 5 und 20 dB Referenz-SNR: 1x1 mit $h=1$, gut konditioniertes 2x2 mit $\mathrm{diag}(\sqrt{2},\sqrt{2})$ und Rang-1-2x2 mit ausschließlich Einsen. Zusätzlich führt es den gut konditionierten Kanal mit explizitem Identitäts-Precoding und mit einem 2x1-Precoder zur Auswahl der ersten TX-Antenne aus. Es misst realen UDP-Durchsatz über SpectrumWifiPhy und gibt zusätzlich PER, Stream-SINR, EESM-SINR und Mindestkanalrang aus. Der Aufruf muss ein positives --eesmBetaLinear liefern. Der im Regressionstest verwendete Wert 2 ist nur ein expliziter Formel-Sentinel; bei diesen flachen Kanälen ist das Ergebnis betaunabhängig und stellt keine Produktionskalibrierung dar.

Die deterministische Referenzausgabe lautet:

- 1x1: Rang 1, Stream-/Effective-SINR 20,000 dB, PER 0,000016 und 68,1408 Mbit/s;
- gut konditioniertes 2x2: Rang 2, beide Stream-SINRs und Effective-SINR 20,000 dB, PER 0,000016 und 134,3904 Mbit/s;
- Rang-1-2x2: Rang 1, beide Stream-SINRs und Effective-SINR -0,043214 dB, PER 1 und 0 Mbit/s;
- Identitäts-Precoding 2x2: dieselben analytischen SINR-, EESM- und PER-Ergebnisse wie der offene Vollrangpfad und 133,4784 Mbit/s im sequenziellen Netzwerkregressionslauf;
- 2x1-Streamauswahl auf dem 2x2-Kanal: Rang 1, Stream-/Effective-SINR 23,010300 dB, PER 0 und 68,1504 Mbit/s. Der 3,0103-dB-Unterschied zum offenen Einzelstreamvergleich folgt aus der vollen Gesamtleistung für $N_s=1$, nicht aus zusätzlicher Precoderleistung.

5.9 FESTES PRECODING DER UNTERPHASE 3.1

MimoPrecodingMatrix ist ein unveränderlicher, dimensionssicherer Werttyp für

$$
  \mathbf W[k]\in\mathbb C^{N_\mathrm{tx}\times N_s},
  \qquad
  1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx}).
$$

Seine flache Reihenfolge ist verbindlich subcarrier/tx/stream:

$$
  j(k,t,s)=(kN_\mathrm{tx}+t)N_s+s.
$$

Der frequenzselektive Konstruktor erwartet deshalb genau
$(N_\mathrm{sc}N_\mathrm{tx}N_s)$ komplexe Koeffizienten. FrequencyFlat
erwartet genau $(N_\mathrm{tx}N_s)$ Werte in tx/stream-Reihenfolge und
expandiert sie beim Erzeugen des Werttyps explizit auf alle (N_sc)
Subcarrier. Intern existiert damit kein implizites Broadcasting und jeder
Zugriff besitzt dieselben drei Dimensionen. Alle Real- und Imaginärteile
müssen endlich sein.

Für jeden Subcarrier gilt verpflichtend

$$
  \lVert\mathbf W[k]\rVert_F^2
  =\sum_{t=0}^{N_\mathrm{tx}-1}\sum_{s=0}^{N_s-1}|w_{t,s}[k]|^2
  =N_s.
$$

Die absolute Prüftoleranz beträgt (10^{-10}). Eine verletzte Norm wird
abgelehnt und nicht stillschweigend korrigiert. Mit
$(\mathrm E[\mathbf x\mathbf x^H]=\mathbf I)$ lautet das erweiterte
Signalmodell

$$
  \mathbf y[k]=
  \sqrt{\frac{P}{N_s}}\,\mathbf H[k]\mathbf W[k]\mathbf x[k]
  +\mathbf n[k].
$$

Damit bleibt (P) die gesamte Sendeleistung und

$$
  \mathbf H_\mathrm{eff}[k]=\mathbf H[k]\mathbf W[k]
  \in\mathbb C^{N_\mathrm{rx}\times N_s}.
$$

Der bestehende MMSE-Kern wird ohne neuen Detektortyp auf
$(\mathbf H_\mathrm{eff})$ angewendet. Mit
$(\alpha=\rho/N_s)$ gilt weiterhin

$$
  \mathbf E[k]=
  \left(\mathbf I_{N_s}+\alpha
  \mathbf H_\mathrm{eff}^H[k]\mathbf H_\mathrm{eff}[k]\right)^{-1},
  \qquad
  \gamma_s[k]=\frac{1}{E_{s,s}[k]}-1.
$$

SuMimoLinkModel, HeSuMimoLinkAbstraction und der Spectrumadapter besitzen
dafür explizite Precoding-Pfade. Ihre bisherigen Überladungen ohne W[k]

bleiben bestehen und rechnen unverändert direkt mit H[k]. Insbesondere nimmt der bytegleiche 1x1-IPC-Vertrag kein neues Feld auf: Sionna liefert weiterhin ausschließlich H[k], W[k] existiert nur in ns-3.

SionnaMimoConfiguration enthält optional precodingMatrix. Validate prüft vor jeder Helper-Zustandsänderung die TX-Dimension, die Gleichheit von maxNss und W-Spalten sowie $N_s\leq N_\mathrm{rx}$. Apply ersetzt den Adapterzustand entweder durch den vollständig validierten Precoder oder löscht ihn explizit, sodass erneutes Anwenden einer Phase-2-Konfiguration kein altes W[k] zurücklässt. Am Laufzeitpfad werden zusätzlich CFR-Subcarrierzahl und beide CFR-Antennendimensionen geprüft, bevor ErrorRateModel- oder Empfangszustand verändert wird. WifiTxVector::GetNss muss bei vorhandenem W[k] exakt dessen Spaltenzahl, andernfalls weiterhin exakt N_tx sein.

Der Vertrag implementiert ausschließlich benutzergegebenes festes Precoding. Er leitet W[k] nicht aus H[k] ab und führt weder MRT/MRC noch SVD, CSI-Rückmeldung, Beamforming oder MU-MIMO ein.

6. HIGH-LEVEL-API-VERTRAG BIS UNTERPHASE 3.1

6.1 TYPEN, BESITZ UND DEFAULTS

SionnaMimoHelper ist ein wertbasierter, simulationsunabhängiger Konfigurationshalter. Sein Konstruktor erzeugt einen gültigen SISO-Vertrag; der Konstruktor mit SionnaMimoConfiguration und SetConfiguration prüfen eine vollständige Kopie, bevor sie den gespeicherten Zustand ersetzen. Eine fehlgeschlagene Änderung lässt den bisherigen Zustand damit unverändert. Alle Validierungsfehler sind std::invalid_argument mit einem SionnaMimoHelper-Präfix und benötigen weder Simulator, Node noch eine ZMQ-Verbindung.

SionnaMimoArrayConfiguration enthält Zeilen, Spalten, vertikalen und horizontalen Abstand in Wellenlängen, Elementmuster und Polarisation. Die repräsentierten Sionna-Muster sind isotrop, dipole, half-wave dipole und TR 38.901. Vertikale und horizontale Polarisation erzeugen je Element einen, duale Polarisation zwei Antennenports. Für Zeilenzahl R, Spaltenzahl C und Polarisationsfaktor $p\in\{1,2\}$ gilt deshalb

$$
  N_\mathrm{array} = R C p.
$$

SionnaMimoOrientation speichert Azimut, Elevation und Rollwinkel in Grad. Die SionnaMimoConfiguration führt beide Arrays, den Modus, TX-/RX-Antennenzahl, maximale NSS, Empfänger, Normalisierung, Empfängerrauschzahl, Effective-SINR-Modell, explizite EESM-Betas und eine Map von ns-3-Node-ID auf Orientierung sowie optional MimoPrecodingMatrix zusammen. Die festgeschriebenen Defaults sind:

- Modus SISO, eine TX-Antenne, eine RX-Antenne und maximale NSS eins;
- TX-Array 1 x 1, Abstände 0,5 Wellenlängen, TR-38.901-Muster, vertikale Polarisation;
- RX-Array 1 x 1, Abstände 0,5 Wellenlängen, Dipolmuster, vertikale Polarisation;
- keine explizite knotenspezifische Orientierung; der API-Lesezugriff liefert dafür neutrale Nullwinkel, während Apply kein IPC-Feld erzeugt und der Server deshalb für alte und neue 1x1-Aufrufe exakt seine bisherige interne Orientierung [0, -180, 0] beibehält;
- linearer MMSE-Empfänger und gemeinsame Unit-Average-Power-Normalisierung über alle Subcarrier und Antennenpaare gemäß Abschnitt 4.3;
- 7 dB Empfängerrauschzahl entsprechend dem ns-3.40-WifiPhy-Default;
- EESM als HE-Effective-SINR-Modell, aber weiterhin keine impliziten

MCS-Betas;
- kein Precoding. Dadurch nehmen alle bisherigen 1x1- und offenen Mehrstromkonfigurationen weiterhin exakt den Phase-2-Pfad.

SetNodeOrientation, ClearNodeOrientation und GetNodeOrientation verwalten die knotenbezogenen Winkel. SetEesmBeta, HasEesmBeta und GetEesmBeta verwalten explizite lineare Betas für HE MCS 0 bis 11. SetPrecodingMatrix, ClearPrecodingMatrix, HasPrecodingMatrix und GetPrecodingMatrix verwalten den optionalen festen Precoder atomar. Die gespeicherte Konfiguration bleibt ein wertbasierter Vertrag und verändert Simulationsobjekte erst mit Apply.

6.2 UNTERSTÜTZTE BETRIEBSARTEN

GetSupportedModes und IsModeSupported melden genau zwei akzeptierte Modi:

1. SISO mit genau einer TX- und einer RX-Antenne sowie NSS eins;
2. SU_MIMO_SPATIAL_MULTIPLEXING mit linearem MMSE-Empfänger. Ohne W[k] bleibt genau ein Stream je TX-Antenne aktiv; mit explizitem festem W[k] bestimmt dessen Spaltenzahl N_s.

SU_BEAMFORMING und MU_MIMO sind im Enum ausschließlich als stabile Namen für zukünftige Phasen repräsentiert. Festes benutzergegebenes Precoding wird bewusst nicht in SU_BEAMFORMING umbenannt, weil es keine aus H[k] abgeleitete Strahlformung implementiert. Validate weist beide reservierten Modi weiterhin eindeutig zurück. Ebenso
werden der repräsentierte Zero-Forcing-Empfänger, eine unnormalisierte CFR und Mutual Information als PER-wirksames Effective-SINR-Modell abgelehnt. Die MI-Kompression aus Abschnitt 5.4 bleibt eine Diagnosemetrik und wird nicht fälschlich als kalibrierte PER-Abbildung angeboten.

6.3 ZENTRALE KONSISTENZPRÜFUNG

Validate erzwingt gemeinsam die Grenzen von Sionna, ns-3-Wi-Fi und dem Phase-1-Abstraktionsmodell:

- Arraydimensionen und Abstände sind endlich und positiv; die abgeleitete Portzahl stimmt exakt mit numTxAntennas beziehungsweise numRxAntennas überein;
- TX-/RX-Antennenzahlen und maxNss liegen im ns-3.40-Bereich 1 bis 8;
- SISO besitzt exakt die Dimensionen 1 x 1 und maxNss eins;
- SU-MIMO ohne W[k] erfüllt weiterhin $2\leq N_\mathrm{tx}\leq N_\mathrm{rx}\leq8$ und $N_\mathrm{ss,max}=N_\mathrm{tx}$;
- SU-MIMO mit W[k] erfüllt $1\leq N_s=N_\mathrm{ss,max}\leq \min(N_\mathrm{tx},N_\mathrm{rx})$. TX- und Streamdimension stimmen exakt mit der Matrix überein; Subcarrierzahl, Koeffizienten und Leistungsnorm sind bereits Bestandteil des unveränderlichen MimoPrecodingMatrix-Vertrags;
- Azimut und Rollwinkel liegen endlich in [-180, 180] Grad, Elevation endlich in [-90, 90] Grad;
- die Rauschzahl ist endlich und nicht negativ;
- jedes vorhandene EESM-Beta gehört zu HE MCS 0 bis 11 und ist endlich und positiv.

ValidateForHeMcs ergänzt die nutzungsbezogene Vollständigkeitsprüfung: Für Open-Loop-SU-MIMO muss das konkret verwendete MCS ein explizites Beta besitzen. SISO verlangt wegen des unveränderten skalaren Phase-1-Pfads weiterhin kein Beta. Damit werden keine unbelegten Kalibrierwerte eingeführt.

6.4 GEMEINSAMER INTEGRATIONSAUFRUF

Apply erhält einen vorhandenen SionnaHelper, getrennte
SpectrumWifiPhyHelper-Instanzen für Sender und Empfänger sowie den
SionnaSpectrumPropagationLossModel-Adapter. Die Trennung der beiden
PHY-Helper hält auch \(N_\mathrm{tx}\ne N_\mathrm{rx}\) darstellbar. Apply
validiert zuerst erneut den vollständigen gespeicherten Vertrag und verändert
danach ausschließlich die explizit übergebenen Objekte:

```
    SionnaMimoHelper mimo(configuration);
    mimo.Apply(sionna, txPhy, rxPhy, spectrumModel);
```

Die Zuordnung ist eindeutig:

- txArray und rxArray werden auf die bestehenden
ConfigureTransmitArray-/ConfigureReceiveArray-Aufrufe abgebildet;
- numTxAntennas beziehungsweise numRxAntennas setzen das Antennenzahlattribut
des jeweiligen SpectrumWifiPhyHelper;
- maxNss setzt auf beiden Endpunkten MaxSupportedTxSpatialStreams und
MaxSupportedRxSpatialStreams;
- receiverNoiseFigureDb setzt sowohl WifiPhy::RxNoiseFigure als auch den
expliziten Rauschzahlwert des MIMO-Spectrumadapters;
- die vollständige EESM-Map ersetzt atomar die bisherige Adapterkonfiguration,
sodass beim erneuten Anwenden keine veralteten MCS-Betas zurückbleiben;
- precodingMatrix ersetzt den festen Adapter-Precoder vollständig. Fehlt der
optionale Wert, wird ein zuvor gesetztes W[k] gelöscht und der Phase-2-Pfad
wiederhergestellt.

Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung und EESM
benötigen keinen zusätzlichen Laufzeitselektor. Das Vorhandensein von W[k]
wählt ausschließlich die explizite \(H[k]W[k]\)-Überladung. Eine
abweichende Empfänger-, Normalisierungs- oder Effective-SINR-Auswahl wird
weiterhin vor Apply abgelehnt.

6.5 KNOTENORIENTIERUNG IM IPC- UND SERVERPFAD

NodeInfo besitzt das neue optionale Feld orientation mit Azimut, Elevation und
Rollwinkel in Grad. SionnaHelper trägt nur explizit konfigurierte
Knotenorientierungen ein. Der Python-Server prüft endliche Werte und dieselben
Winkelgrenzen wie die High-Level-API, konvertiert Grad mit

$$
\boldsymbol\theta_\mathrm{rad}
= \frac{\pi}{180}\boldsymbol\theta_\mathrm{deg}
$$

und verwendet die drei Eulerwinkel beim Anlegen jedes Transmitter- und
Receiver-Objekts im klassischen sowie im Look-ahead-Pfad. Das ausgelagerte
PopulateMimoConfiguration schreibt Array- und Orientierungsfelder sowohl für
Start als auch für den bytegenauen Integrationstest.

Fehlt orientation, verwendet der Server weiterhin exakt die bisherige
Konstante [0, -180, 0]. Der Wert wird bewusst nicht als neue API-Orientierung
interpretiert, sondern nur zur vollständigen Verhaltenskompatibilität alter
Nachrichten beibehalten. Ein alter SionnaHelper-Aufruf konfiguriert weder Array-
noch Orientierungsfelder und erzeugt deshalb weiterhin die bisherigen
1x1-Arrays und die bisherige Serverausrichtung.

6.6 BENUTZBARKEIT, BEISPIELE UND MIGRATION

Für eine 1x1-Verbindung reicht der Defaultvertrag. Er behält SISO, je eine
TX-/RX-Antenne, NSS eins, 7 dB Rauschzahl, die bisherigen Arraymuster und den

skalaren Legacy-Empfangspfad bei. Deshalb ist für diesen Pfad kein EESM-Beta erforderlich:

```
    SionnaMimoConfiguration configuration;
    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Das kleinste Vollrang-2x2-Beispiel verwendet zwei vertikale Elemente je Array, zwei offene Spatial Streams und ein vom Anwender kalibriertes Beta für den gewählten HE-MCS:

```
    SionnaMimoConfiguration configuration;
    configuration.mode = SionnaMimoMode::SU_MIMO_SPATIAL_MULTIPLEXING;
    configuration.txArray.numColumns = 2;
    configuration.rxArray.numColumns = 2;
    configuration.numTxAntennas = 2;
    configuration.numRxAntennas = 2;
    configuration.maxNss = 2;
    configuration.eesmBetaLinear.emplace(mcs, betaFromCalibration);

    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Eine feste 2x1-Streamauswahl auf einem 2x2-Kanal ergänzt ausschließlich maxNss und den normalisierten Precoder:

```
    configuration.maxNss = 1;
    configuration.precodingMatrix = MimoPrecodingMatrix::FrequencyFlat(
    numSubcarriers, 2, 1, {{1.0, 0.0}, {0.0, 0.0}});

    SionnaMimoHelper mimo(configuration);
    mimo.ApplyForHeMcs(mcs, sionna, txPhy, rxPhy, spectrumModel);
```

Für frequenzselektives Precoding wird stattdessen der MimoPrecodingMatrix-Konstruktor mit \(N_\mathrm{sc}N_\mathrm{tx}N_s\) Koeffizienten in sc/tx/stream-Reihenfolge verwendet. Beide Varianten durchlaufen dieselbe Validierung und denselben Adapterpfad.

ApplyForHeMcs führt zuerst ValidateForHeMcs aus. Ein ungültiger MCS oder ein fehlendes SU-MIMO-Beta bricht daher mit std::invalid_argument und einem SionnaMimoHelper-Präfix ab, bevor SionnaHelper, die beiden PHY-Helper oder der Spectrumadapter verändert werden. Apply bleibt für vorhandenen Code erhalten, wenn der verwendete MCS an dieser Stelle noch nicht bekannt ist.

Das Programm ns3_wifi_sumimo_minimal.cc enthält die vollständigen helperkonfigurierten 1x1-, offenen Vollrang-2x2-, Identitäts-Precoding- und 2x1-Streamauswahlszenarien. Es verbindet sie über eine feste normalisierte CFR mit dem echten UDP-/SpectrumWifiPhy-Pfad und behält zusätzlich den Phase-1-Rang-1-Vergleich:

```
    ./ns3 run "ns3_wifi_sumimo_minimal \
    --mcs=5 --referenceSnrDb=20 --packetSize=1200 \
    --eesmBetaLinear=<kalibrierter-linearer-Wert>"
```

Der feste Kanal ersetzt in diesem reproduzierbaren Beispiel nur die Kanalquelle. Array, Antennenzahlen, NSS, Rauschzahl und EESM werden nicht am Adapter oder PHY gesetzt, sondern ausschließlich mit SionnaMimoHelper.

Für die Migration eines bestehenden Programms werden die manuellen Aufrufe ConfigureTransmitArray, ConfigureReceiveArray, die PHY-Attribute Antennas,

MaxSupportedTxSpatialStreams, MaxSupportedRxSpatialStreams und RxNoiseFigure sowie SetReceiverNoiseFigureDb/SetEesmBeta am Spectrumadapter durch genau eine SionnaMimoConfiguration und ApplyForHeMcs ersetzt. SionnaHelper::Configure, SetMode, Start und Destroy, der PropagationCache sowie die normale SpectrumChannel-Verdrahtung bleiben unverändert. Ein Programm, das weiterhin nur SionnaHelper verwendet, nimmt damit unverändert den bisherigen 1x1-Defaultpfad.

Funktional unterstützt der Stand bis Unterphase 3.1 SISO, offenen SU-MIMO mit einem Stream je TX-Antenne und festes benutzergegebenes Precoding mit $1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})$. Linearer MMSE, gemeinsame Unit-Average-Pair-Power-Normalisierung der CFR und EESM bleiben unverändert. Typische eindeutige Fehler sind:

  - SionnaMimoHelper: open-loop SU-MIMO uses one stream per TX antenna bei widersprüchlichem maxNss;
  - SionnaMimoHelper: only SISO and open-loop SU-MIMO spatial multiplexing are supported für Beamforming oder MU-MIMO;
  - SionnaMimoHelper: open-loop SU-MIMO requires an explicit EESM beta for HE MCS ... bei fehlender Kalibrierung;
  - SionnaMimoHelper: maximum NSS must equal the fixed-precoding stream dimension bei widersprüchlicher Helper-Konfiguration;
  - WifiTxVector NSS must equal the transmitted stream dimension, falls WifiTxVector und W-Spalten beziehungsweise der alte offene TX-Pfad widersprüchlich sind.

Die Enumwerte für Beamforming und MU-MIMO bleiben ausschließlich reservierte API-Namen. Unterphase 3.1 implementiert weder MRT/MRC, SVD, CSI-Feedback, Beamforming noch MU-MIMO.

7. VALIDIERUNG DER UNTERPHASEN 1.1 BIS 3.1

Die native Suite sionna enthält nun 23 schnelle Fälle:

1. API-Defaulttest für den vollständigen legacy-kompatiblen SISO-Vertrag, leere EESM-Kalibrierung, fehlendes W[k] und exakt zwei gemeldete Betriebsarten.
2. Gültiger 2x2-Vertrag mit dual polarisiertem TX-Element, knotenspezifischer Orientierung, explizitem Beta, Identitäts-Precoding, frequenzselektiver 2x1-Streamauswahl sowie zulässigem 2-TX-/1-RX-/1-Stream-Grenzfall.
3. Zentrale Negativmatrix für nicht endliche, unvollständige, widersprüchliche oder nicht unterstützte API-Konfigurationen einschließlich Precoding-TX-/NSS-Dimension, Beamforming, MU-MIMO und fehlendem MCS-Beta.
4. Manuell und über SionnaMimoHelper konfigurierte 1x1-Integration mit bytegleicher Array-/Orientierungs-IPC-Nachricht, identischer installierter SpectrumWifiPhy-Antennen-/NSS-Konfiguration, gleicher Rauschzahl und EESM-Kalibrierung sowie exakt gleichen MMSE-, EESM- und PER-Ergebnissen.
5. Derselbe Integrationsvergleich für eine vollständige offene 2x2-Konfiguration ohne W[k].
6. Manuell gesetztes und durch SionnaMimoHelper angewendetes frequenzselektives komplexes 2x1-W[k] liefern dieselben Koeffizienten, installierten NSS, Post-MMSE-SINRs, EESM-SINR und PER.
7. Layouttest eines 2 x 2 x 2-CFR-Tensors mit eindeutig nummerierten Koeffizienten.
8. Precoding-Vertragstest für sc/tx/stream-Reihenfolge, explizite Frequenzflach-Expansion, Frobenius-Leistung sowie Dimensions-, Endlichkeits- und Normierungsfehler.
9. Reversible dB-/Linear-Umrechnung einschließlich 10 dB gleich Faktor zehn.
10. Link-Budget-Test für Sendeleistung, Pfadverlust, B, T und NF sowie die erwartete Halbierung der SNR bei Verdopplung von B oder T.

11. Flacher orthogonaler 2x2-Kanal: EESM erhält identische Tone-/Stream-SINRs unabhängig vom positiven Test-Beta.
12. Frequenzselektiver SISO-Kanal: EESM stimmt mit der analytischen Zweitonformel für das konfigurierte Beta überein.
13. SISO-Parität zur direkten ns-3.40-ErrorRateModel-Abfrage für HE MCS 0, 3, 7 und 11, neun SNR-Punkte von -5 bis 35 dB sowie 32, 400 und 1458 Byte. Das sind 108 Kombinationen; MCS 11 prüft auch den TableBased-Fallback auf Yans.
14. Monotone PER auf einem normierten frequenzselektiven Vier-Ton-Kanal für dieselben vier MCS und drei Paketgrößen bei 26 SNR-Punkten von -10 bis 40 dB. Die 312 geprüften Werte bilden je Konfiguration eine nicht steigende Folge mit einem echten PER-Abfall.
15. Exakte Reproduzierbarkeit von Post-MMSE-SINR, EESM-SINR und PER über 20 Wiederholungen eines komplexen frequenzselektiven 2x2-Kanals.
16. SISO-Test mit $h=2+j$ und $\rho=10$; erwartet wird $\gamma=50$.
17. Orthogonaler 2x2-Kanal $\mathbf H=\mathbf I$ und $\rho=20$; jeder Stream erhält $\gamma=10$ und $\eta=2\log_2(11)$.
18. Zwei diagonale Subcarrier diag(1,2) und diag(2,1) bei $\rho=8$; erwartet werden (4,16), (16,4) und die analytische MI-Kompression.
19. Fester-Precoding-Referenztest für komplexes SISO, Identitätsparität, indexgenaues komplexes $H[k]W[k]$, identische MMSE-Ausgabe zum vorberechneten effektiven Kanal und 2x2 mit einem ausgewählten Stream.
20. Mindestkanalrang eins beziehungsweise zwei für Rang-1- und orthogonale 2x2-Kanäle über alle Subcarrier.
21. Spectrumadapter-Test für exaktes Rauschleistungs- und AWGN-Diversity-Compensation-Mapping sowie SISO-, Identitäts- und 2x1-Precoding ohne doppelte Anwendung von Rauschen, Leistung oder Arraygewinn.
22. Deserialisierung einer alten Nachricht ohne Dimensionsfelder als $N_\mathrm{sc}\times1\times1$.
23. Protocol-Buffer-Rundlauf einer synthetischen 2 x 2 x 2-CFR mit den komplexen Werten 1-j bis 8-8j und indexgenauer Prüfung.

Die fünf reinen Python-Vertragstests prüfen Standard- und explizite Arraykonfiguration, bytegleiche 1x1-Serialisierung, die sc/rx/tx-Reihenfolge einer synthetischen 2x2-CFR, die gemeinsame MIMO-Normalisierung sowie die legacy-kompatible und explizite Knotenorientierung. Diese Tests benötigen weder Raytracing, ZMQ noch eine GPU. Die analytischen Vergleiche des numerischen Kerns verwenden weiterhin eine absolute Toleranz von 1e-10; dies gilt auch für Identitäts-Precoding und synthetische $H[k]W[k]$-Referenzen. Die Link-Budget-/EESM-Vergleiche verwenden 1e-12.

Der reale Server-Integrationstest konfiguriert 1x2-PlanarArrays an TX und RX, führt Sionna RT auf der CPU aus und prüft die Form 2 x 2 x 256, 1024 komplexe Werte und mittlere normierte Leistung eins. Die übrigen sieben Servertests decken den unveränderten 1x1-Pfad, Mobilität, Kohärenzzeit und Mehrnutzer-Anfragen ab.

Die ExampleAsTest-Suites enthalten fünf Integrationsprozesse. Der Positivlauf führt 1x1, Vollrang-2x2, Rang-1-2x2, Identitäts-Precoding und 2x1-Streamauswahl über UDP/SpectrumWifiPhy aus. Er prüft die analytischen Metriken exakt und die gemessenen UDP-Raten gegen deterministische Referenzwerte. Drei frühe Negativläufe prüfen widersprüchliches NSS, ein fehlendes MCS-Beta im MCS-bezogenen Apply-Aufruf und den reservierten Beamforming-Modus. Ein weiterer Negativlauf installiert die 2x2-Konfiguration über SionnaMimoHelper, liefert aber eine 1x1-CFR und erreicht nachweislich den Dimensionsguard im SpectrumWifiPhy-Adapter.

7.1 AUSGEFÜHRTE CPU-VALIDIERUNG

Am 1. August 2026 wurden nach Abschluss der Unterphase 3.1 folgende Prüfungen auf einem CPU-System erfolgreich ausgeführt:

- inkrementeller Build der geänderten Bibliotheks-, nativen Test- und Beispielziele unter GCC 13.3; zusätzlich wurde das unveränderte P2P-Legacybeispiel neu gelinkt
- native ns-3-Suite sionna: 23 von 23 Fällen bestanden; die Integrationsfälle vergleichen manuelle und helperbasierte 1x1-/2x2-Konfigurationen ohne W[k] sowie frequenzselektives komplexes 2x1-W[k] ohne einen laufenden ZMQ-/Sionna-Server
- SISO-Paritätsmatrix: alle 108 Kombinationen stimmen innerhalb 1e-12 mit der direkt aufgerufenen ns-3-Fehlermodellschnittstelle überein
- frequenzselektive Monotonitätsmatrix: alle 312 PER-Werte bestanden
- Reproduzierbarkeit: 20 identische 2x2-Aufrufe mit exakt gleichen Ergebnissen
- ExampleAsTest sionna-example-sumimo-minimal: ein von einem Fall bestanden; Vollrang-2x2 erreicht 134,3904 statt 68,1408 Mbit/s bei 1x1, Rang-1-2x2 erreicht bei PER 1 erwartungsgemäß 0 Mbit/s; Identitäts-Precoding reproduziert die analytischen Vollrangwerte und die 2x1-Auswahl erreicht 23,010300 dB bei voller Gesamtleistung
- ExampleAsTest sionna-example-mimo-helper-configuration-negative: drei von drei Fällen bestanden; ungültiges NSS, reservierter Beamforming-Modus und fehlendes MCS-Beta brechen jeweils früh mit Status 2 und eindeutiger Meldung ab
- ExampleAsTest sionna-example-mimo-helper-runtime-negative: ein von einem Fall bestanden; eine helperkonfigurierte 2x2-PHY mit 1x1-CFR erreicht den erwarteten Dimensionsguard im echten SpectrumWifiPhy-Laufzeitpfad
- reine Python-Vertrags- und Import-Suite: fünf von fünf Fällen in 0,001 s bestanden
- gesamte Sionna-CPU-Serversuite: acht von acht Fällen in 270,483 s bestanden; der reale 2x2-Fall lief in 3,0752 s, lieferte 2 x 2 x 256 und erfüllte die Normalisierung innerhalb 1e-6
- P2P-End-to-End-Test example-sionna-sensing im unveränderten 1x1-Modus: drei CFR-Antworten, zwei empfangene UDP-Pakete zu je 1024 Byte und Cache-Hit-Rate 0,993056

Der ältere Beispielpfad example-sionna ist in der unveränderten Basis defekt: Er startet standardmäßig Mode 3 mit Submode 1, berechnet aber keinen Look-ahead-Zeitpunkt. Dadurch wird kein Receiver angelegt und Sionna meldet ValueError: Scene has no receivers. Der explizite P2P-Beispielpfad ist davon nicht betroffen. Eine Korrektur gehört in eine eigene kleine Baseline-Subphase.

8. BEKANNTE GRENZEN UND NÄCHSTE ENTSCHEIDUNGSPUNKTE

- SionnaMimoHelper überträgt eine gemeinsame TX- und RX-Geometrie für die jeweilige integrierte Verbindung. Knoten können unterschiedliche Orientierungen besitzen; heterogene Arraygeometrien pro Knoten sind im aktuellen Vertrag nicht repräsentiert.
- Nicht explizit orientierte Knoten behalten die alte Serverorientierung [0, -180, 0]. Explizite Winkel werden in beiden Servermodi an Sionna übertragen.
- Der integrierte MIMO-Pfad unterstützt HE SU ohne Precoding mit einem Open-Loop-Stream je TX-Antenne oder mit festem, benutzergegebenem \(W[k]\). Nicht-HE-Kontrollrahmen verwenden das erste Antennenpaar; kanaladaptives Precoding, Beamforming und MU-MIMO sind nicht implementiert.
- Zeitvariable externe Interferenz wird nach der MMSE-/EESM-Abstraktion durch InterferenceHelper skalar berücksichtigt. Eine frequenz- und antennenabhängige Interferenzkovarianz im MMSE-Detektor ist noch nicht modelliert.
- SionnaMimoHelper übernimmt die Rauschzahl einmalig und setzt sie identisch am Adapter sowie an beiden PHY-Helpern. Code ohne High-Level-API muss beide Werte weiterhin manuell konsistent halten, da ns-3.40 das PHY-Attribut nicht lesbar macht.
- Es existiert absichtlich keine mitgelieferte HE-EESM-Beta-Tabelle.

  Frequenzselektive absolute PER-Prognosen benötigen vor der wissenschaftlichen
  Nutzung eine dokumentierte Link-Level-Kalibrierung. Ohne diese Konfiguration
  bricht das Modell früh ab.
  - TableBasedErrorRateModel ist eine AWGN-Referenz. Seine SNR-/PER-Kurven und
  Paketlängenextrapolation stammen aus ns-3; ihre Grenzen gelten unverändert.
  - Perfekte RX-CSI ist zunächst bewusst idealisiert.
  - Ohne W[k] gilt weiterhin genau ein Stream je TX-Antenne. Mit festem W[k]
  sind kontrollierte Streamauswahl und
  \(1\leq N_s\leq\min(N_\mathrm{tx},N_\mathrm{rx})\) implementiert.
  - Die MI-Kompression bleibt eine Diagnosemetrik; nur die explizit konfigurierte
  EESM-Ausgabe wird an ErrorRateModel übergeben.
  - Die bestehende Cache-Schlüsselbildung nimmt Reziprozität an. Für
  nicht-reziprokes Precoding müssen Richtung und Konfiguration Teil des
  Schlüssels werden.

Unterphase 3.1 ist abgeschlossen: Frequenzflaches und frequenzselektives
festes Precoding, Streamauswahl, Leistungsnormierung, Helper-/Adapterintegration
und End-to-End-Regressionen sind vorhanden. MRT/MRC, SVD, CSI-Feedback,
kanaladaptives Beamforming und MU-MIMO wurden bewusst nicht begonnen.

9. AUSFÜHRUNG

Vom ns-3.40-Verzeichnis:

```
    ./ns3 configure --enable-examples --enable-tests
    ./ns3 build
    ./test.py -s sionna
    ./test.py -s sionna-example-sumimo-minimal
    ./test.py -s sionna-example-mimo-helper-configuration-negative
    ./test.py -s sionna-example-mimo-helper-runtime-negative
    ./ns3 run "ns3_wifi_sumimo_minimal --eesmBetaLinear=2"
```

Python-Vertrag und Server vom Verzeichnis contrib/sionna/model/ns3sionna:

```
    conda run -n sionna-venv python -m unittest -v tests.test_mimo_data_contract
    conda run -n sionna-venv python -m unittest -v tests.unit_tests
```

PDF neu erzeugen:

```
    conda run -n sionna-venv python
    contrib/sionna/tools/render_model_pdf.py
    contrib/sionna/ns3sionna_mimo_model.md
    contrib/sionna/ns3sionna_mimo_model.pdf
```

Das Rendering-Skript benötigt nur Matplotlib und läuft im vorhandenen
CPU-Environment.